

# CSCI 112

## Programming with C

### LECTURE: 2

J. L. Pach

# OUTLINE:

- Boolean logic
- Introduction to C: Hello World
- Indentation & parentheses
- Comments
- Symbolic Name
  - Variables
  - Arrays
  - Labels
  - Functions
- ASCII

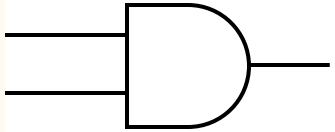
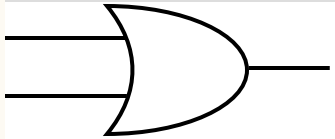
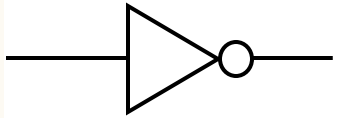
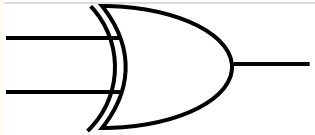
# BOOLEAN

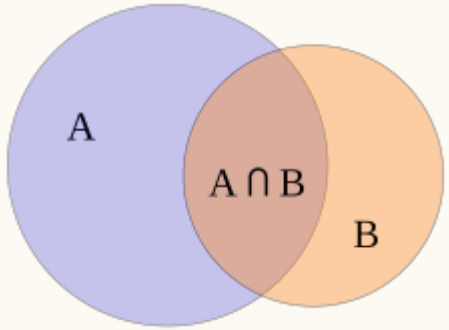
logic

# GEORGE BOOLE

- **Boolean logic** is a fundamental subset of algebra used for binary variables, representing **true** and **false** values through operations like `AND`, `OR`, and `NOT`. It's essential in computer science for circuit design, programming, and digital computations.
- **George Boole** was a 19th-century English mathematician who laid the groundwork for Boolean logic, revolutionizing mathematics and computer science. Fascinatingly, Boole died from pneumonia after being soaked in a rainstorm and treated with controversial cold-water therapy. His life and work are detailed in his biography, "*The Life and Work of George Boole*", which offers great insights into his contributions to modern technology.

# BOOLEAN LOGIC

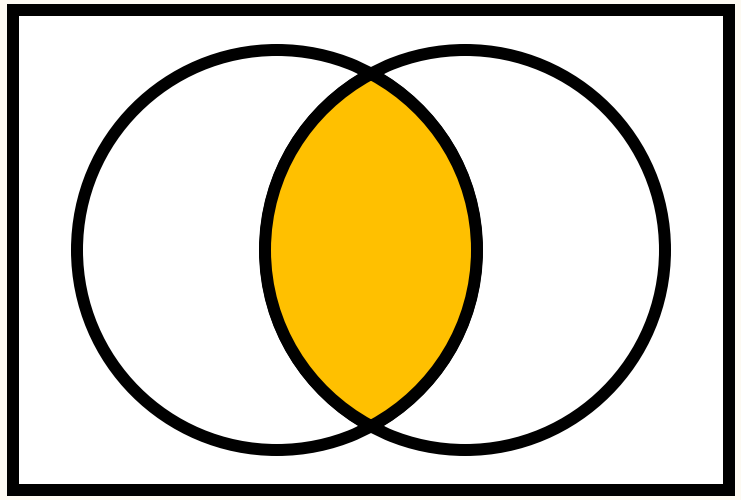
| Symbol | Logical operation | Operator | Notation |
|---|---|---|---|
|  | Conjunction ("product" of x and y) | AND | x ∧ y |
|  | Disjunction ("sum") of x and y) | OR | x ∨ y |
|  | Negation ("the opposite") | NOT | ¬x |
|  | Exclusive OR | XOR | x ⊻ y |




***Watch out!** The conjunction symbol ^ from logic is **XOR** in C, not AND!

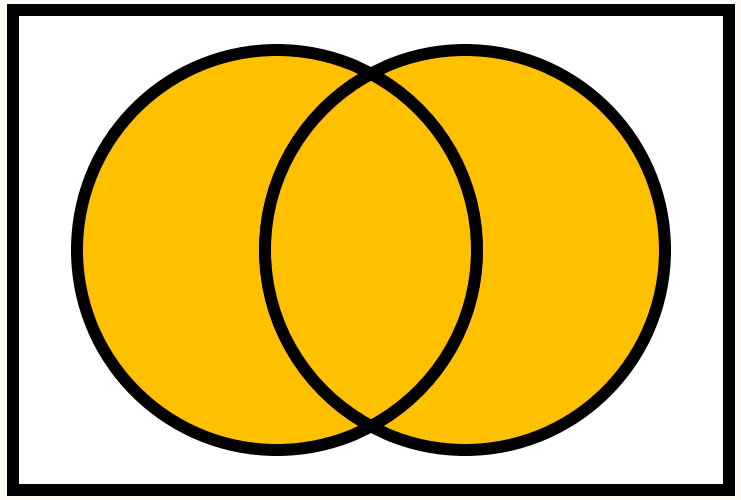
# AND GATE

| x | y | x ∧ y |
|---|---|---|
| 0 | 0 | 0 |
| 0 | 1 | 0 |
| 1 | 0 | 0 |
| 1 | 1 | 1 |



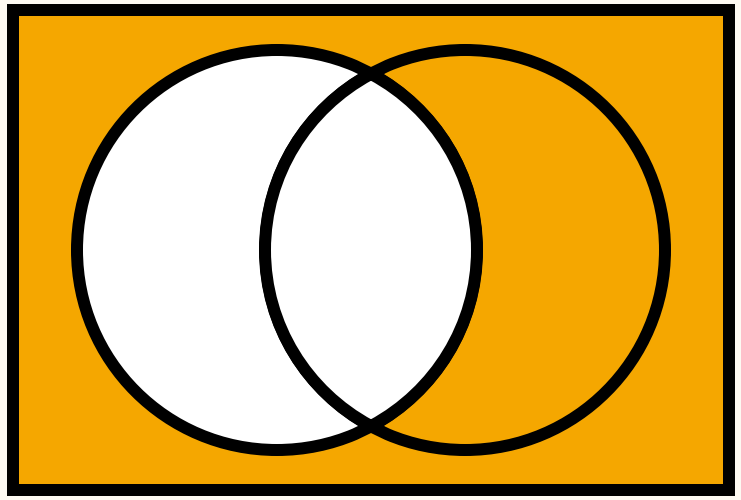
# OR GATE

| x | y | x V y |
|---|---|---|
| 0 | 0 | 0 |
| 0 | 1 | 1 |
| 1 | 0 | 1 |
| 1 | 1 | 1 |



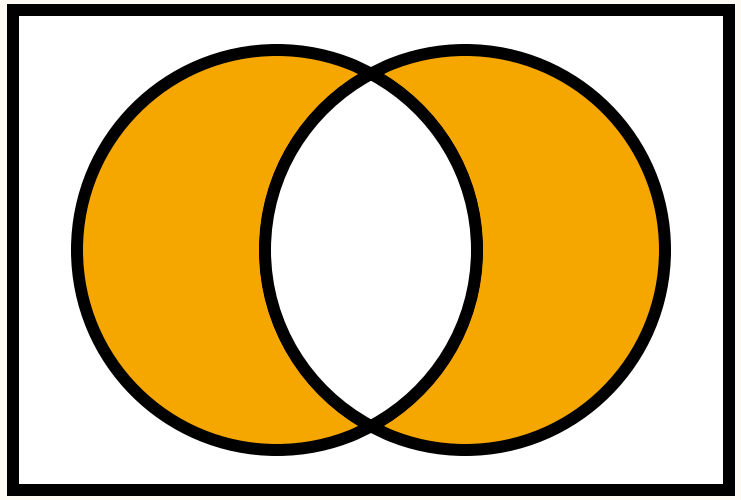
# NOT GATE

| x | ¬x |
|---|---|
| 0 | 1 |
| 1 | 0 |



# XOR GATE

| x | y | x ⊻ y |
|---|---|---|
| 0 | 0 | 0 |
| 0 | 1 | 1 |
| 1 | 0 | 1 |
| 1 | 1 | 0 |



# INTRODUCTION

## to C: Hello World

```c
#include <stdio.h>

int main()
{
  char string[12] = "Hello world";
  printf("%s", string);
  return 0;
}
```

# HELLO WORLD

- Text preceded by `#` is a preprocessor section, the first line gives access to standard input and output functions, this is a header.
- Next, we have the main function defined, which returns an integer value. The `{}` brackets start and end the body of the main function.
- The `printf` function displays the string `Hello world` on the console.

**Result**

```
Hello world
```

# `PRINTF()` IS A FUNCTION AND HAS TWO SIDES:

```c
char string[12] = "Hello world";
printf("%s", string);
```



- Left side `"%s"` – format string:
  - This is the *printing plan* — text + special placeholders (`%s`, `%d`, `%f`, etc.) that tell `printf()` how to insert values into specific places.
  - In `"%s"`, the `%` means *this is not just plain text*, but an instruction:
    - *take the data from the right side and display it in the format specified on the left side* — in this case, as a string.

# PRINTF() IS A FUNCTION AND HAS TWO SIDES:

```c
char string[12] = "Hello world";
printf("%s", string);
```



- Right side string – data to insert:
  - This is the variable or value that `printf()` will *plug in* where `%s` is. In this example, `printf()` reads all the characters of the string from the first to the last.

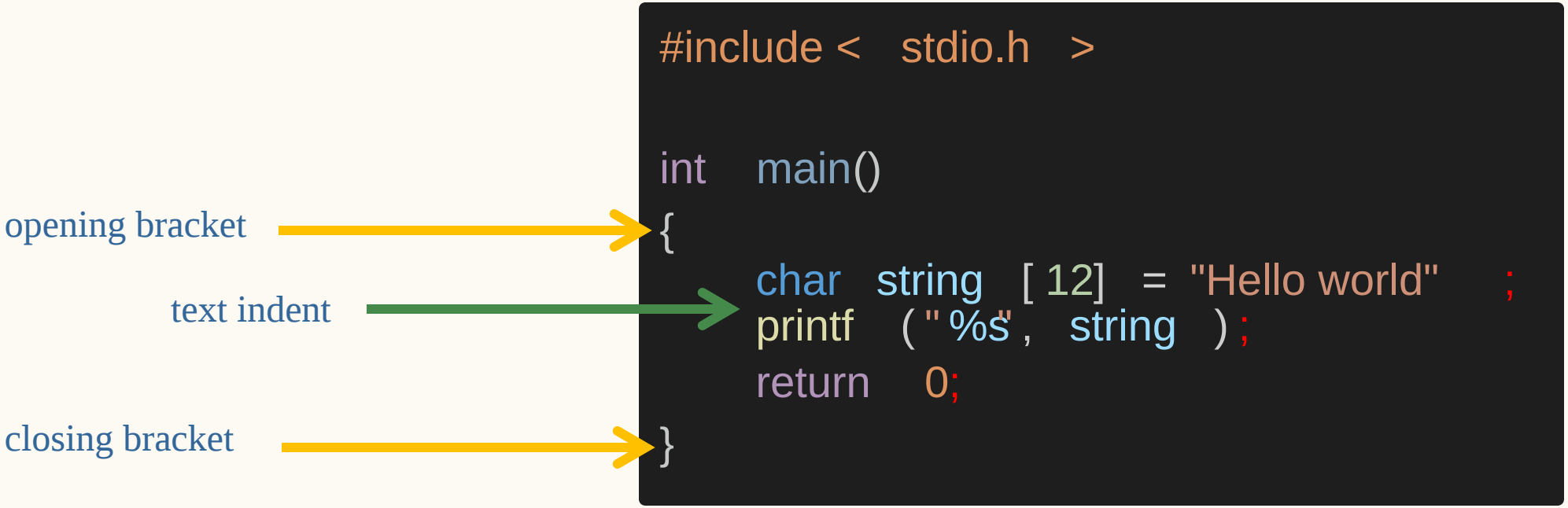
# SEMICOLON;

```
#include <stdio.h> int main(){char string[12] = "Hello world"; printf("%s", string); return 0;}
```



- In the C programming language statements for the compiler (interpreter in Python) is separated by a semicolon `;`.
- Therefore, in C, you can write an entire program on one line...

# INDENTATION & PARENTHESES




- Proper indentation is essential for making C code readable,
- Formatting is mandatory in Python, but not required here – We are talking about the compiler, because in our classes formatting is mandatory in order to get a positive grade at all.

# INDENTATION & PARENTHESES BUT...

```c
#include <stdio.h>

int main()
{
	char string[12] = "Hello world";
	printf("%s", string);
	return 0;
}
```



```c
#include <stdio.h>

int main(){
	char string[12] = "Hello world";
	printf("%s", string);
	return 0;
}
```



# COMMENTS

```c
int main()
{

  char string[12] = /* inline comment */  "Hello world";

  printf("%s", string);  /* comment behind the line */

  /* a comment
  composed of
  a few lines */

  /* comment */ int x = 2; // comment

    return 0; // single-line comments
}
```



# SYMBOLIC

Name

# WHAT IS AN ADDRESS?

**Address of University:**

- 1300 W Park St, Butte, MT 59701



# WHAT IS AN ADDRESS?

**Address of University:**

- 1300 W Park St, Butte, MT 59701




- Address is an identifier (**symbolic name**) of location
- A place to locate what we refer to

# WHAT IS A NAME OF VARIABLE?

```c
int main()
{
  char* text = "Hello world\n";

  printf(text);

    return 0;
}
```



- a variable name is a **symbolic name**, and when translating the code, the compiler will replace the variable name with the memory location of the data associate
- every **symbolic name** is an alias for a memory location (address) except for preprocessor instructions

# SYMBOLIC NAMES WILL BE USED IN:

- **Variables**: Symbolic names will be used to identify and refer to data stored in variables. This allows for more meaningful and descriptive code compared to using arbitrary names or identifiers.
- **Arrays**: Symbolic names will be used to identify collections of related data elements. Arrays can be used to store multiple values of the same data type.
- **Functions**: Symbolic names will be used to identify and call functions within the program. This helps in organizing and structuring the code, making it easier to understand and maintain.
- **Labels***: Symbolic names will be used to mark specific locations or points within the program code. These labels can be used for various purposes, such as control flow statements, data references, or error handling.

# RESTRICTIONS ON SYMBOLIC NAME

- The first character must be a letter, underscore `_`, or special character `@`, `#`, `$`*
- The remaining characters can be letters, digits, underscores
- cannot contain spaces
- cannot be C language keywords ( for example. `for`, `while`, etc.)

* *Only variable or function names*

# SYMBOLIC NAMING CONVENTION

**camelCase**  **vs** **snake_case** 

- `camelCase` starts each word with a capital letter, except for the first word.
  - For example, `thisIsCamelCase`.
- `snake_case` uses underscores to separate words and all letters are lowercase.
  - For example, `this_is_snake_case`.

Regardless of the specific coding style, it's common practice to start variable and function names with a lowercase letter. When using `snake_case`, we use underscores to separate words, like `my_variable`.

Constants, which are values that don't change, are usually written in all uppercase letters, such as `MAX_VALUE`

# SYMBOLIC NAMES WILL BE USED IN:

- **Variables**: Symbolic names will be used to identify and refer to data stored in variables. This allows for more meaningful and descriptive code compared to using arbitrary names or identifiers.
- Arrays
- Functions
- Labels

# VARIABLES

# DECLARING AND INITIALIZING VARIABLES 1/3

```c
int main()
{
  int p;        /* Declaration of variable p with a size of 4 bytes */
  int q, r, s;  /* Simultaneous declaration of variables q, r, s using "," */
  q = 2;        /* Assignment of value to variable q - initialization */
  r = q = s;    /* Assignment of values to q and s based on r */
  int t = 3;    /* Declaration and initialization on the same line */

  return 0;
}
```



# DECLARING AND INITIALIZING VARIABLES 2/3

```c
int main()
{
  char v;             /* Variable v of integer type with a size of 1 byte */
  short int w;         /* Variable w of integer type with a size of 2 bytes */
  long int x;          /* Variable x of integer type with a size of 4 bytes */
  short y;            /* Shorthand declaration for short int */
  long z;             /* Shorthand declaration for long int */

  return 0;

  // short == short int
  // long  == long int == int
}
```



# DECLARING AND INITIALIZING VARIABLES 3/3

```c
int main()
{
  float  a = 3.16f;    /* Variable a of floating-point type with a size of 4 bytes */
  double b = a * 3.0;  /* Variable b of floating-point type with a size of 8 bytes */
  long double d;       /* Variable d of floating-point type with a size of 12 bytes */

  /* Note: short float, long float, and short double do not exist in C */
}
```



# DECLARING AND INITIALIZING VARIABLES

- Any variable must be declared before use.
- Unlike Python, C requires explicit type declaration for variables*
- to write integers we use the type `char` (1B), `short integer` (2B) or `long integer` (4B)
- write real numbers `float` (4B), `double` (8B) or `long double` (12B)
- An uninitialized variable takes the random value

*there is an auto keyword, but it is not allowed in the entire programming course!

# TWO WORDS ABOUT FLOATING-POINT REPRESENTATION

Operations on real numbers are recorded with only a certain degree of precision, and therefore there is a very high probability that the result of `a + b – c` will not be the same as `a - c + b` ! This means that using real numbers requires careful consideration.

# POINTERS ARE TREATED AS FIRST-CLASS DATA TYPES

```
int main()
{
  short int p;        /* Declaration of variable p with a size of 2 bytes */
  short int * q = &p; /* Declaration and initialization of pointer q with a size of 4 bytes
                      (even though it points to short int) */
  float*r;            /* Declaration of pointer r to float  */
  char* s;            /* Declaration of pointer s to char  */
  int t, *v;          /* Declaration of variable t and pointer v */
  short int* w, z;    /* Declaration of pointer w to short int and variable z */

/* "Note that the variable type is determined by the position of the asterisk ('*') in the declaration.
Only the variable directly following the asterisk is considered a pointer. */
}
```

We can create a pointer to **any** data type using the `*` operator between the existing data type and the symbolic name. Unary Operator `&` returns memory locations. A pointer is a reference to a specific memory location

# SUMMARY OF MEMORY SIZE OF DATA TYPES

| Type | Memory size in bytes / bits |
|---|---|
| `char` | 01 Bytes / 08 bits |
| `bool`* | 01 Bytes / 08 bits |
| `short int` | 02 Bytes / 16 bits |
| `long int` | 04 Bytes / 32 bits |
| `float` | 04 Bytes / 32 bits |
| `double` | 08 Bytes / 64 bits |
| `long double` | 12 Bytes / 96 bits |
| `pointer (char*, short*, long*, int*, float*, double*, long double*, void*, …)` | 04 Bytes / 32 bits |
| `user defined (struct, union, etc. )` | complex |

*from C99

# SYMBOLIC NAMES WILL BE USED IN:

- Variables
- **Arrays**: Symbolic names will be used to identify collections of related data elements.

  Arrays can be used to store multiple values of the same data type.
- Functions:
- Labels

# ARRAYS

# ARRAYS - RESERVES SPACE IN MEMORY

```c
int a[10];
```



- This statement reserves space in memory for 10 integers and creates an '*unchanging address of memory*' that points to the beginning of this array*. You can use this symbolic name to access individual elements of the array using square brackets and the appropriate index.
- The values of array will be undefined, meaning they can hold any random value.

***Array indexing starts from 0.**

# ARRAYS

```
<type> symbolic_name[size];
```

- When you specify the size of an array in square brackets, it is created with that exact size.

```
<type> symbolic_name[] = {value1, value2, value3};
```

- If you omit the size but provide initial values, the compiler counts them and creates an array of that size.

```
<type> symbolic_name[size] = {value1, value2};
```

- If you specify the size but don't initialize all elements, the remaining ones will have indeterminate, unpredictable values.

# THANK

You